

从工具到智能体：AI 使用范式的演进

从工具沿革入手，理解 Claude Code 要解决的问题



- 训练层：任何人都无法触达
- 平台注入：仅 Anthropic 可写
- 系统提示：开发者可写
- 用户消息：你在对话框里输入的

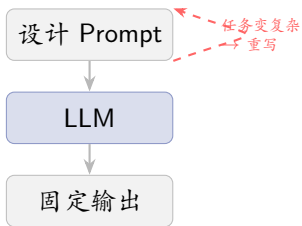
其他客制化的工具 $\approx$ 已经深度编写系统提示层

历史对话 (Assistant Message)

解决复杂任务能力提升



阶段一：Prompt Engineering（2021–2022）



- 通过 API 将 LLM 嵌入预设工作流
- 任务复杂度 $\propto$ Prompt 复杂度
- 模型无法自主决定下一步

→ 瓶颈在人，不在模型

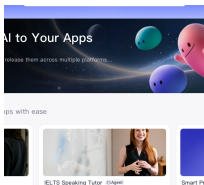
阶段二：使用工具解决复杂任务

为什么需要设计工具？

当任务复杂后，即使写出再复杂的 Prompt 也无法让模型正确完成任务，且每次任务变更都需要重写 Prompt。

工作流编排

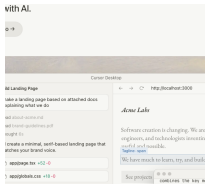
Coze



任务拆分
节点可视化连线

智能辅助

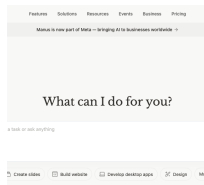
Cursor / Copilot



读写用户工作区
减少输入负担

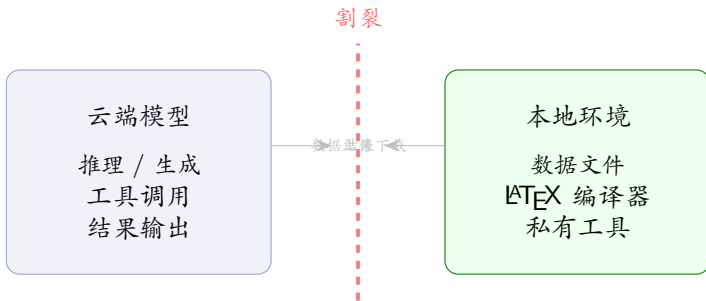
Agent 类

Manus



自主工具调用
多步骤规划

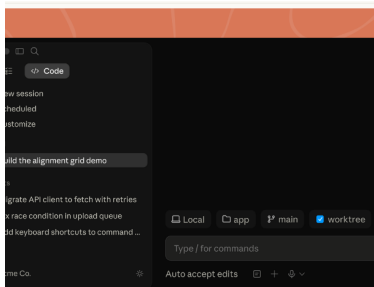
问题所在：模型在云端，数据在本地



典型场景

Manus 写出论文大纲 → 但无法直接调用本地 L^AT_EX 编译器输出 PDF

深度集成：Claude Code（2024+）

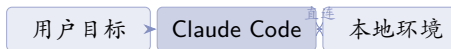


- 用户可自定义框架

→ AI 自主决定读哪些文件

→ AI 自主调用工具

→ 结果直接写回本地工作区



割裂消失，
用户只需给出目标

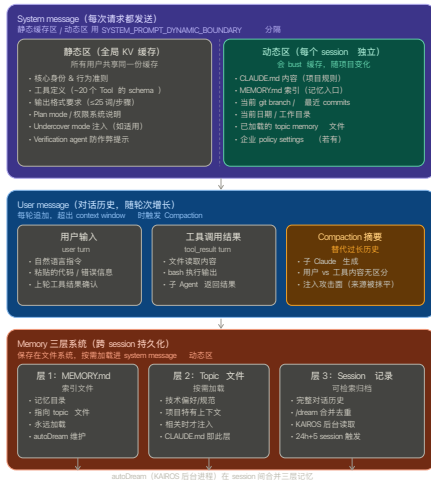
Claude Code 内部如何工作?

为什么 Claude Code 能在本地环境中，
自主读文件、跑命令、跨 session 记住项目上下文?



上下文架构：对照 API 请求的层次

2025 年初 Claude Code 源码泄露，提供了解析这套框架的难得机会



System Prompt

System Message

静态区 (全局 KV 缓存)

- 工具 schema (≈20 个 Tool)
- 核心行为准则 / 角色设定
- 输出格式 (≤25 词 / 步骤)

DYNAMIC_BOUNDARY

动态区 (每 session 独立)

- CLAUDE.md 内容
- git 状态 / 最近 commits
- 当前日期 / 已加载 memory 文件

■ 静态区

所有用户 **共享** 同一份缓存

→ 显著降低每次推理成本

■ 动态区

每次新鲜生成, 会破坏全局缓存

→ 让 Claude 了解你的项目

■ CLAUDE.md = 你的项目说明书

写入动态区, Claude 每次对话都读取

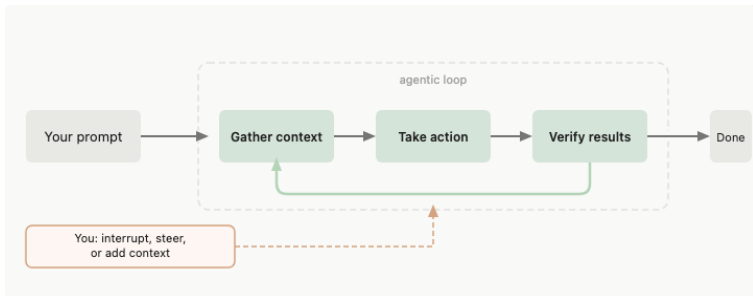
User Message



关键机制

Claude 能"看到"文件，是因为文件内容以工具调用结果的身份注入对话——而不是 Claude 直接访问文件系统

工作逻辑：Agentic Loop



- ① 收集上下文：读文件、搜索代码库
- ② 采取行动：编辑、运行命令
- ③ 验证结果：读输出、跑测试

Tools 是自主性的来源——没有 Tools，Claude 只能回文字；
有了 Tools，每次调用结果反馈回循环，驱动下一步决策

Tools 与其他设计速读

内置 Tools (预设能力)

读写文件	FileRead / FileEdit
执行命令	Bash
搜索代码	Grep / Glob
网络访问	WebFetch
子代理	Agent (subagent)

运行模式

默认	逐步确认
Plan	先探索再给计划
Auto	自动接受所有编辑

可扩展层 (用户可定义)

- **Skills**: 特定任务指南, 基于 Tools 运作
- **MCP**: 外部服务工具接口
- **Hooks**: 事件触发脚本
- **CLAUDE.md**: 项目规则, 每次自动注入

Tools 规定**怎么做** (读 / 写 / 跑),
Skills 规定**做什么** (调哪个 API、遵循哪些约定)

到底如何使用 Claude Code

从下载安装，到配置框架，到实际跑起来——
我踩过的坑和沉淀下来的经验



安装与模型接入

两种安装方式

Native Install (官方站下载)

无功能差异

npm Install (社区镜像)

无法访问外网 → 优先选 npm

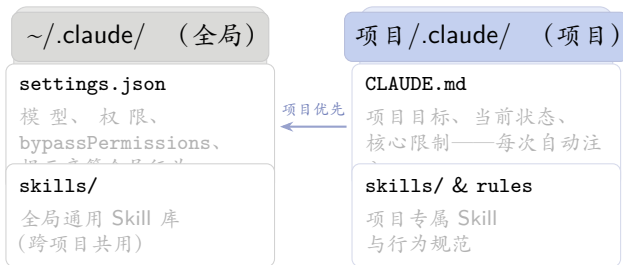
三种模型接入路径

方式	适用场景
官方账号登录	可访问外网
国内 Coding Plan	使用 Qwen / GLM 等
国内中转站	使用国外模型

注意

中转站存在数据泄露风险；
安装遇到环境依赖问题可用社区工具（如 cc switch）

框架骨架：两级配置层



建议

Skill 统一放全局目录，避免各项目分散后难以统一维护；Memory 让 Claude 自行管理，不建议手动修改

五条实操建议

1 先沉淀规范，再写 Skill

先将规律性内容写入 CLAUDE.md；仅当 Claude 反复出错时，才将规范固化为 Skill

2 Skill 保持单一职责

过长的 Skill 会让 Claude 读完后无法正确执行，每个 Skill 只做一件事

3 拆解任务，构建底层通用 Skill

搜索文献、跑回归、画图、排版……底层能力做成通用 Skill，复杂任务组合调用

4 Plan First

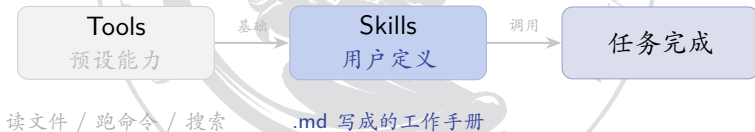
执行前先让 Claude 探索当前任务情况，写出完整执行计划，再开始行动

5 Everything is Prompt

所有框架都可用 Prompt 替代——何时 Plan、如何验证、怎样执行，完全可以自定义

Skills: 给 Claude 配上专属工作手册

Tools 规定**怎么做**（读、写、跑），
Skills 规定**做什么**（用哪个流程、遵循哪些约定）



Skill 的项目结构与制作规范

文件结构



三级渐进加载

1. 元数据 常驻上下文 (≈100 词)
name + description ← 触发机制
2. SKILL.md 正文 触发时加载
完整指令 (建议 ≤500 行), Claude 执行时读取
3. 资源文件 按需加载
scripts 脚本直接执行; references 文档按需读取

制作规范

description 字段是触发的关键:
写清楚何时使用, Claude 才会主动调用此 Skill

直接用 skill-creator Skill 来制作

skill-creator 提供完整的制作流程: 草稿 → 测试 → 评估 → 迭代, 还能自动优化 description 的触发准确率

具体实践：文献搜索 Skill

用户说「帮我做 **ESG** 融资约束的文献综述」 → description 匹配触发 Skill → 选择搜索模式

探索模式

目标：从零构建领域地图

关键词：5-15 组，宽泛

文献量：100-200 篇

搜索层：基础文献 → 前沿 → 工作论文

→ 适合项目启动 / 陌生领域

精准模式

目标：为具体论点补充引用

关键词：1-3 组，精确

文献量：10-30 篇

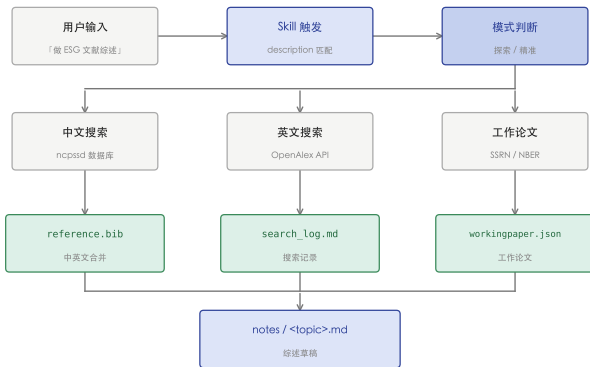
搜索层：顶刊精准 + 按需工作论文

→ 适合写假设 / 引言 / 讨论段

两种模式的提问方式不同

探索模式的关键词要宽 (“ESG performance financing constraints”); 精准模式直接对应论点 (“voluntary ESG disclosure cost of debt”)

从一句话到文献库：完整执行链路



一条指令驱动三路并行搜索、文件写入、PDF 下载，最终生成可引用的综述草稿——这就是 Skill 封装可复用工作流的核心价值



Thanks!

陆知雨

中央财经大学 中国财政发展协同创新中心

zhiyu.lu.econ@icloud.com

<https://zhiyu-lu.com/>